

Chinese Hotel Review Text Analysis

Sentiment + Aspect-Based Text Mining

Shirong Xu

Project Goal

- Dataset: Chinese hotel reviews from ChnSentiCorp
- Objective:
 - Understand customer sentiment
 - Discover important hotel aspects
 - Analyze complaint structures
- Methods:
 - Lexicon matching
 - Frequency analysis
 - Co-occurrence modeling
 - Visualization

Analysis Pipeline

- ① Load dataset
- ② Clean reviews
- ③ Sentiment analysis
- ④ Aspect extraction
- ⑤ Frequency modeling
- ⑥ Visualization + interpretation

Load Dataset

```
DATA_PATH = "ChnSentiCorp_htl_all.csv"

df = pd.read_csv(DATA_PATH)

df = df.dropna(subset=["review"])

df["review"] = df["review"].astype(str)

df["sentiment"] = df["label"].map({
    1: "Positive",
    0: "Negative",
})
```

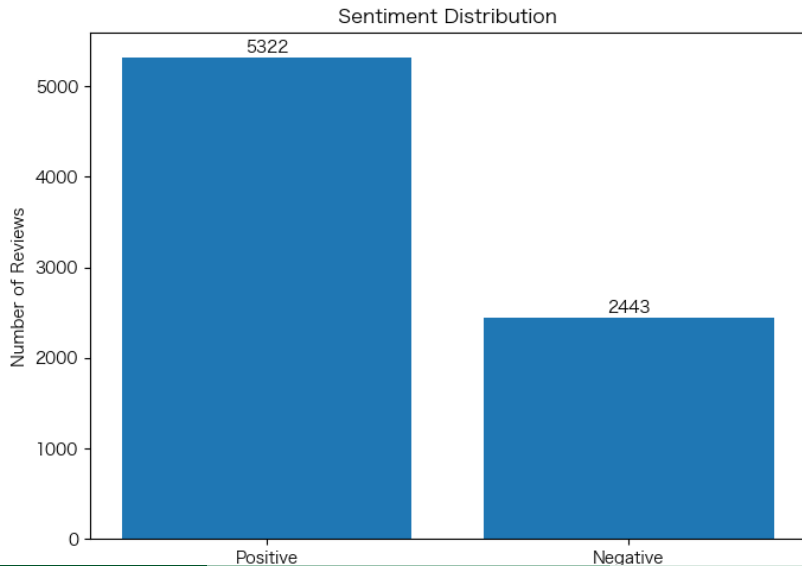
- Remove missing reviews
- Convert text format
- Create sentiment labels

Chinese Character Statistics

```
def chinese_char_count(text):  
    return len(  
        re.findall(r"[\u4e00-\u9fff]", text)  
    )  
  
df["chars"] = df["review"].map(  
    chinese_char_count  
)
```

- Count Chinese characters in each review
- Used to study review length behavior
- Avoid tokenization complexity

Sentiment Distribution

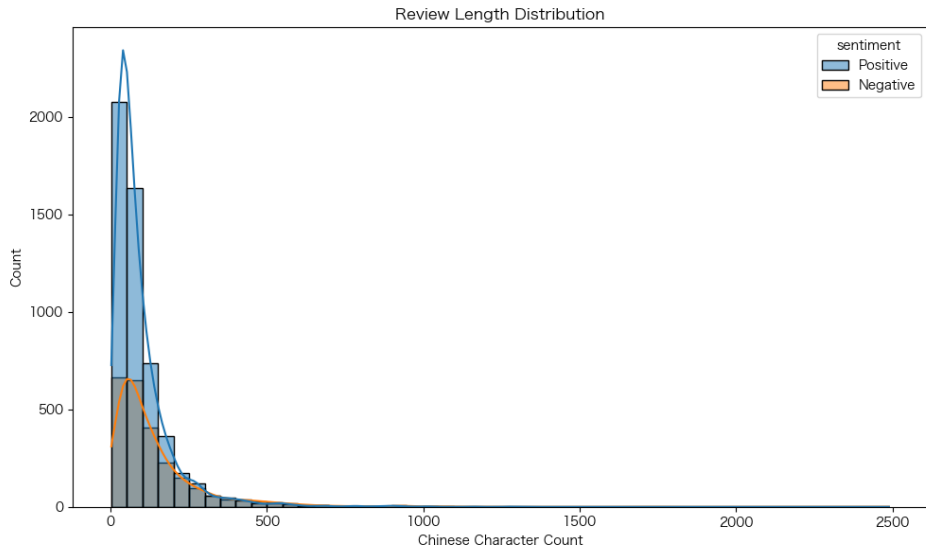


Review Length Visualization

```
sns.histplot(  
    data=df,  
    x="chars",  
    hue="sentiment",  
    bins=50,  
    kde=True,  
    alpha=0.5  
)  
  
plt.title(  
    "Review Length Distribution"  
)
```

- Compare positive vs negative review lengths
- KDE curve shows distribution smoothness

Review Length Distribution



Aspect Dictionary Construction

```
ASPECTS = {"Location": ["位置", "交通", "地铁", "机场", "火车站", "方便", "距离", "附近"],  
           "Room": ["房间", "床", "浴室", "空间", "装修", "窗"],  
           "Cleanliness": ["卫生", "干净", "脏", "异味", "潮湿", "毛巾", "床单"],  
           "Service": ["服务", "前台", "态度", "员工", "热情", "投诉"],  
           "Facilities": ["设施", "空调", "热水", "电视", "wifi", "电梯"],  
           "Price": ["价格", "便宜", "贵", "划算", "实惠", "超值"],  
           "Sleep": ["安静", "吵", "噪音", "隔音", "睡"]}
```

- Dictionary-based aspect mining
- No deep learning required
- Easy to interpret

Keyword Matching Functions

```
def contains_any(text, terms):  
    return any(  
        term in text for term in terms  
    )  
  
def count_terms(text, terms):  
  
    counts = Counter()  
  
    for term in terms:  
  
        n = text.count(term)  
  
        if n > 0:  
            counts[term] += n  
  
    return counts
```

- Detect aspect mentions
- Count sentiment keyword frequencies

Positive vs Negative Word Analysis

```
POSITIVE_TERMS = ["很好", "满意", "推荐", "方便", "干净", "热情",  
                  "舒服", "安静", "划算", "值得"]  
NEGATIVE_TERMS = ["差", "失望", "糟糕", "脏", "吵", "异味", "贵", "投诉", "麻烦"]
```

- Manually selected sentiment lexicon
- Used for interpretable text mining

Frequency Aggregation

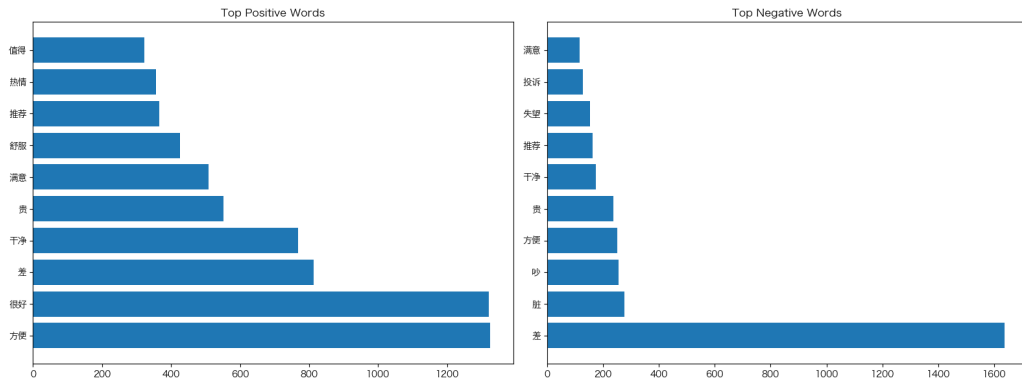
```
positive_counter = Counter()
negative_counter = Counter()

for text in pos_df["review"]:
    positive_counter.update(
        count_terms(text, terms)
    )

for text in neg_df["review"]:
    negative_counter.update(
        count_terms(text, terms)
    )
```

- Aggregate sentiment term counts
- Build corpus-level statistics

Top Word Comparison



- Positive reviews emphasize convenience and cleanliness
- Negative reviews focus on noise and poor conditions

Word Cloud Construction

```
wordcloud = WordCloud(  
    font_path=FONT_PATH,  
    background_color="white",  
    width=1200,  
    height=600  
)  
.generate_from_frequencies(  
    positive_counter  
)
```

- Word size reflects frequency
- Useful for global semantic overview

Positive vs Negative Word Clouds

Positive Review Word Cloud



Positive Reviews

Negative Review Word Cloud



Negative Reviews

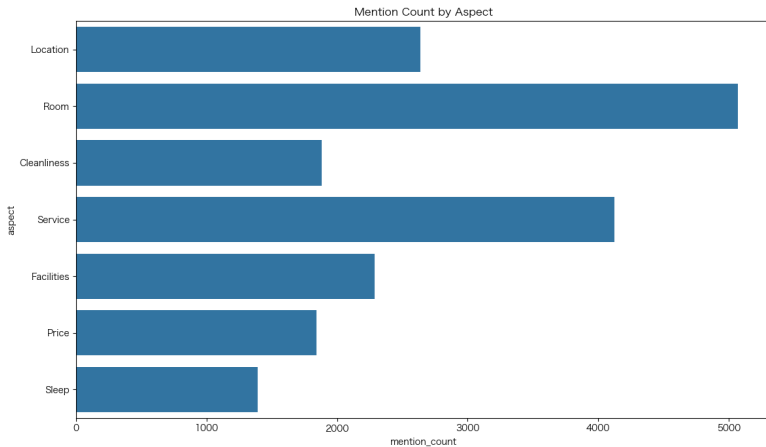
- Positive language is repetitive and simple
- Negative reviews contain richer complaint vocabulary

Aspect-Level Statistics

```
rows = []  
  
for aspect, terms in ASPECTS.items():  
  
    mask = df["review"].map(  
        lambda s:  
            contains_any(s, terms)  
    )  
  
    subset = df[mask]  
  
    mention_count = int(mask.sum())  
  
    neg_rate = (  
        subset["label"] == 0  
    ).mean()
```

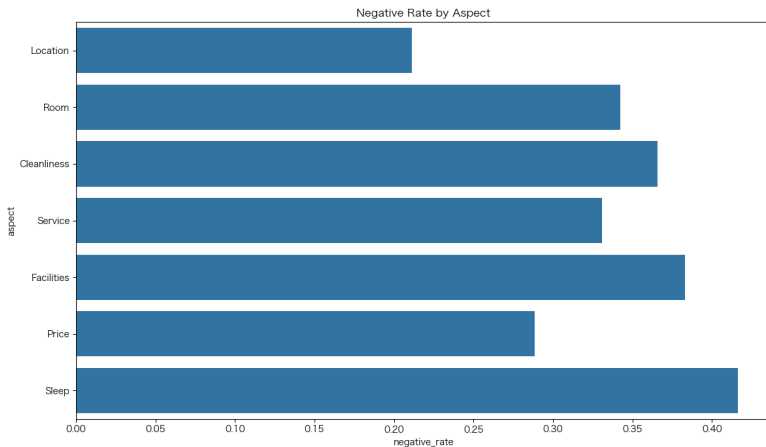
- Compute aspect mention frequency
- Estimate negative review proportion

Aspect Mention Frequency



- Room and location are discussed most frequently
- Service is another dominant factor

Aspect Negative Rate



- Cleanliness has the highest negative rate
- Sleep quality and noise are major complaint sources

Aspect Interpretation

- Location:
 - Usually expected by customers
 - Lower emotional sensitivity
- Cleanliness:
 - Strong driver of dissatisfaction
 - Directly impacts hotel trust
- Service:
 - Emotional interaction matters

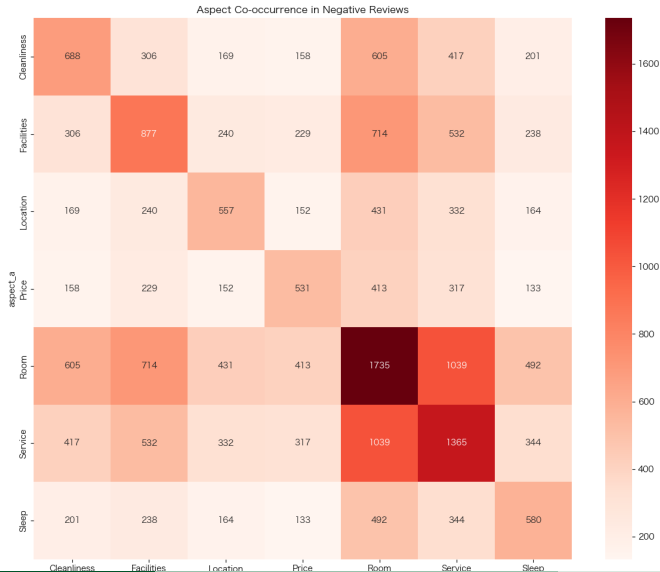
Aspect Co-occurrence Matrix

```
for a in names:
    for b in names:

        rows.append({
            "aspect_a": a,
            "aspect_b": b,
            "count":
                int(
                    (masks[a] & masks[b]).sum()
                )
        })
```

- Measure joint complaints
- Discover coupled hotel problems

Aspect Co-occurrence Heatmap



Network Graph Construction

```
G = nx.Graph()

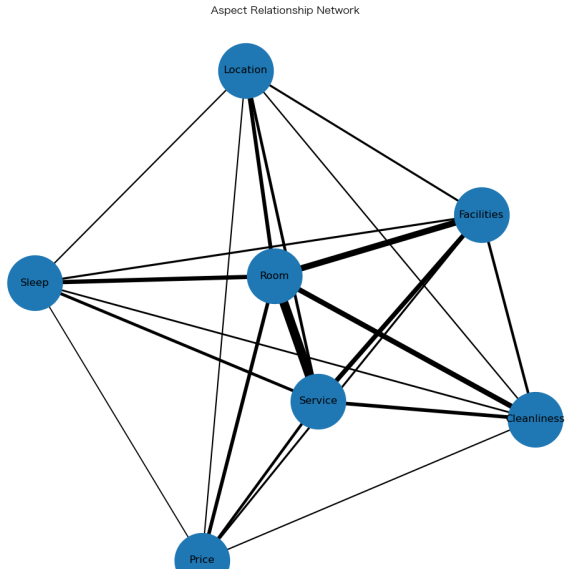
for _, row in co_df.iterrows():

    a = row["aspect_a"]
    b = row["aspect_b"]
    w = row["count"]

    if a != b and w > 100:
        G.add_edge(a, b, weight=w)
```

- Convert co-occurrence into graph structure
- Nodes = aspects
- Edges = complaint relationships

Aspect Relationship Network



Main Findings

- ① Negative reviews are longer and richer
- ② Cleanliness is the strongest risk factor
- ③ Noise and room quality are tightly connected
- ④ Positive reviews use simpler vocabulary
- ⑤ Lexicon-based methods already reveal strong patterns